

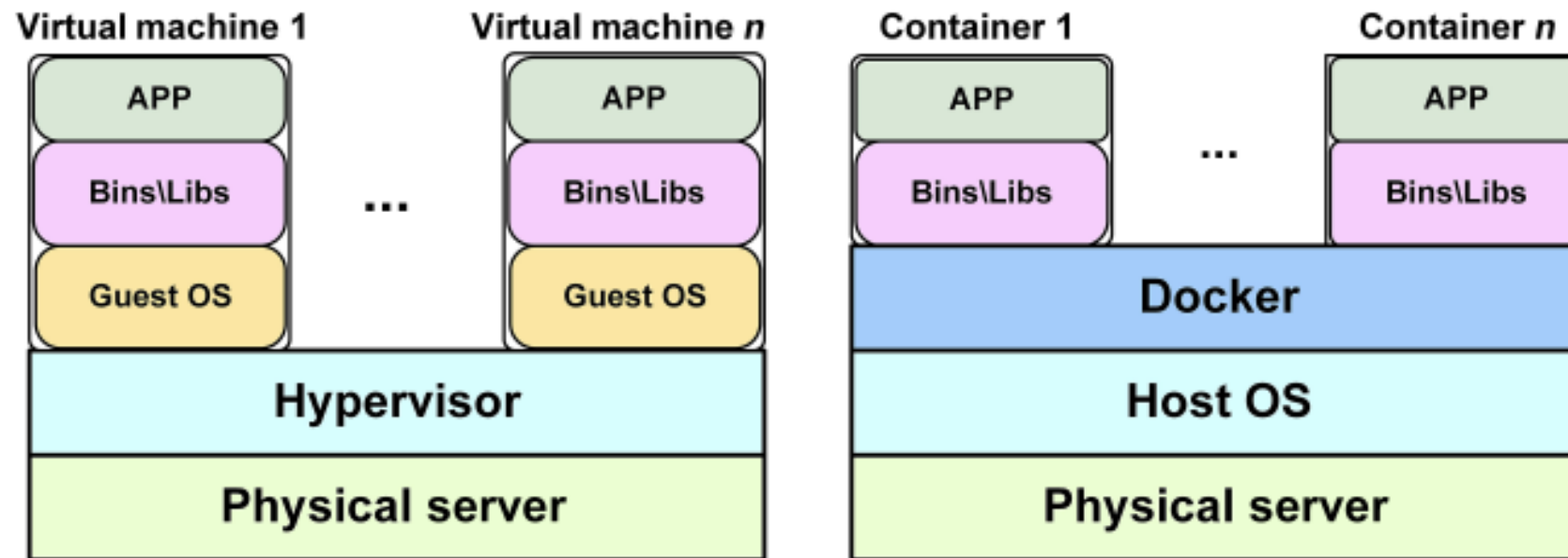


Chapitre 3:

Docker et Kubernetes

Dr. Cheikh Saliou Mbacke Babou

Introduction



Comparaison entre machines virtuelles et conteneurs Docker.

Introduction

- Orchestrateurs
- Mode de fonctionnement
 - Impératif
 - Déclaratif
- Exemple d'orchestrateurs
 - Kubernetes
 - Docker Swarm

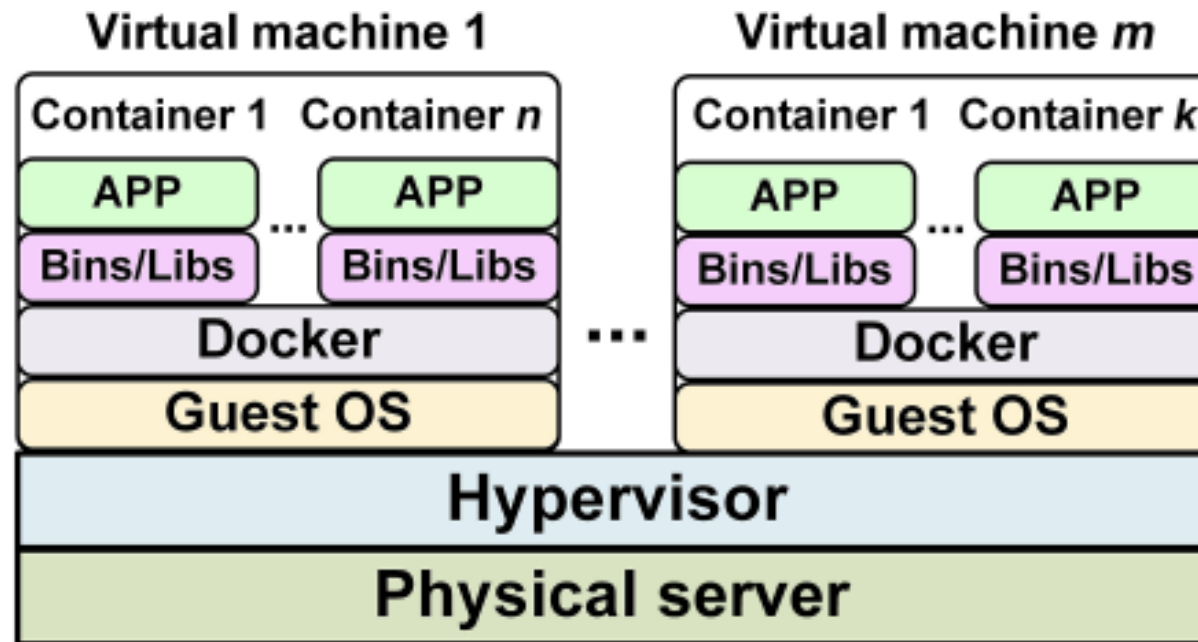
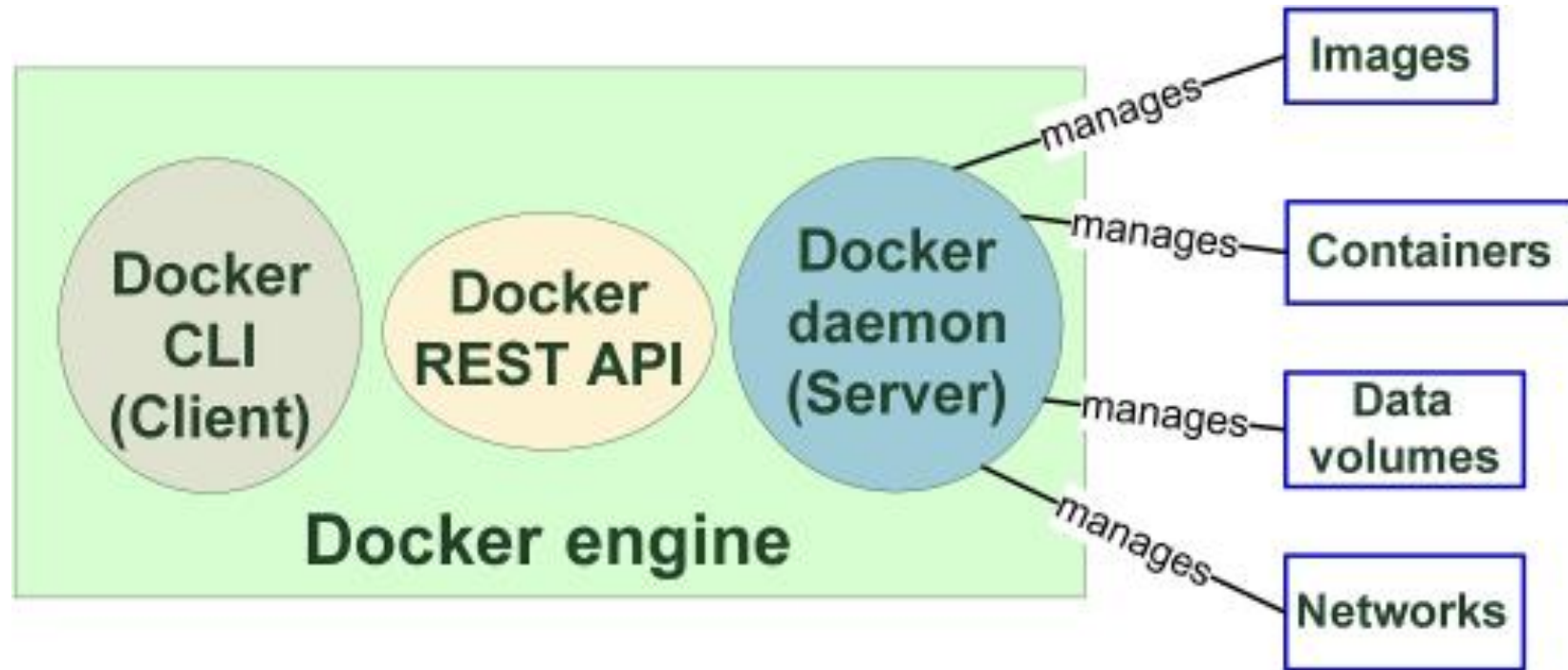


Illustration of Docker containers running on virtual machines

Fondamentaux de Docker

Docker Architecture

- Docker Engine



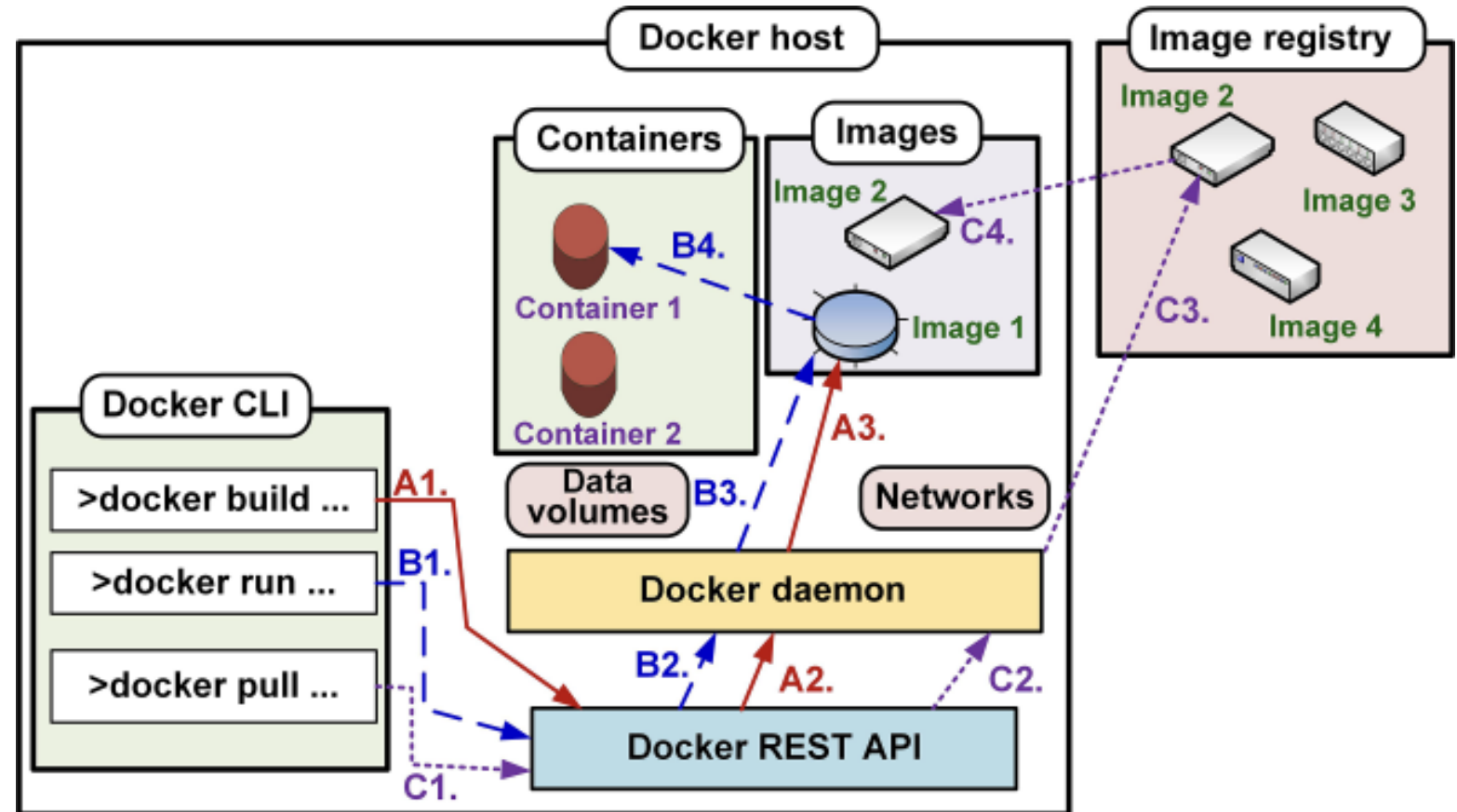
Composants constitutifs du moteur Docker.

Fondamentaux de Docker

Docker Architecture

- Architecture Docker de haut niveau

- ☐ Processus de construction d'une image
- ☐ Processus d'exécution d'un conteneur à partir d'une image locale
- ☐ Processus de téléchargement d'une image depuis un registre d'images

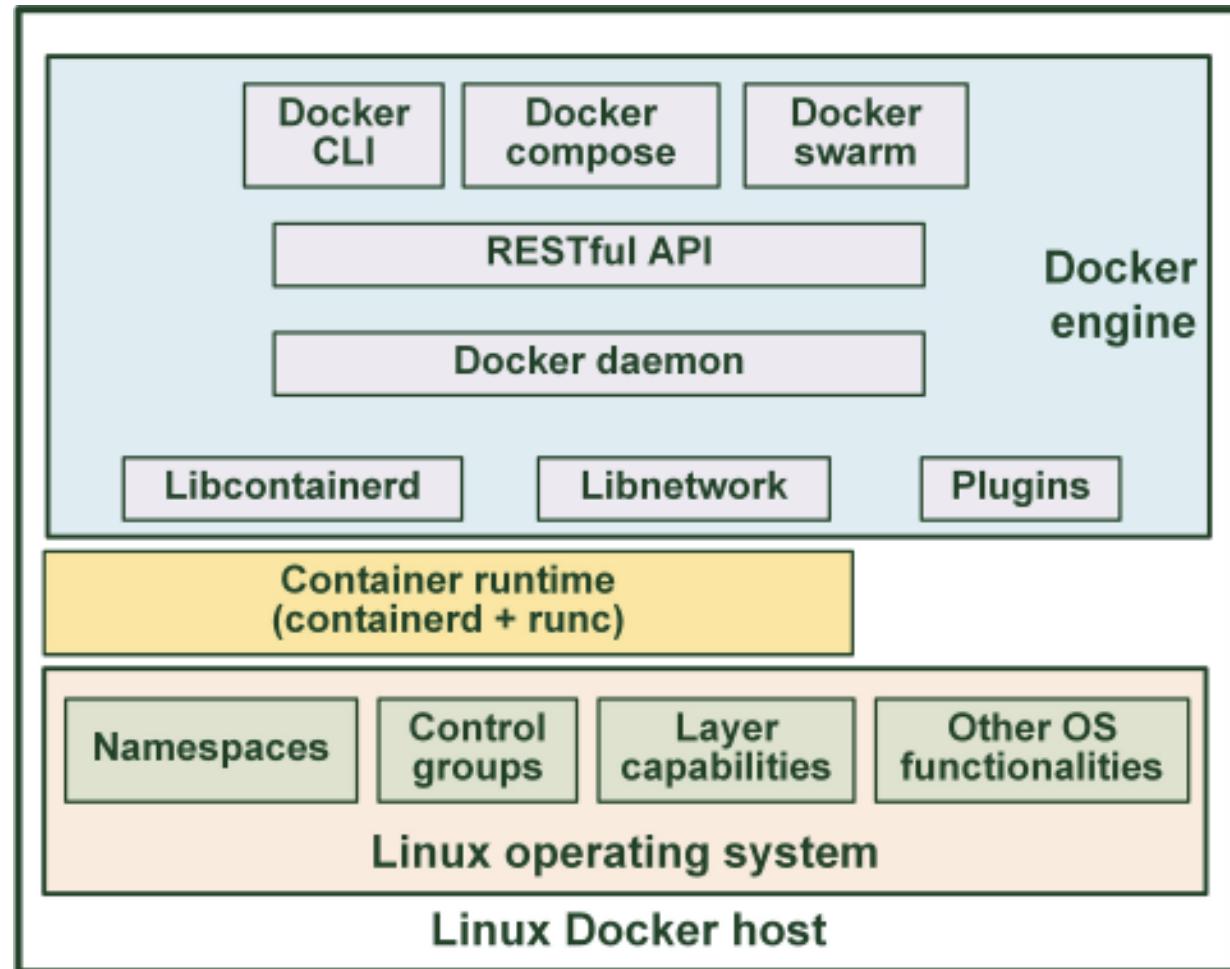


Une vue d'ensemble de l'architecture Docker avec certains de ses flux de travail

Fondamentaux de Docker

Docker Architecture

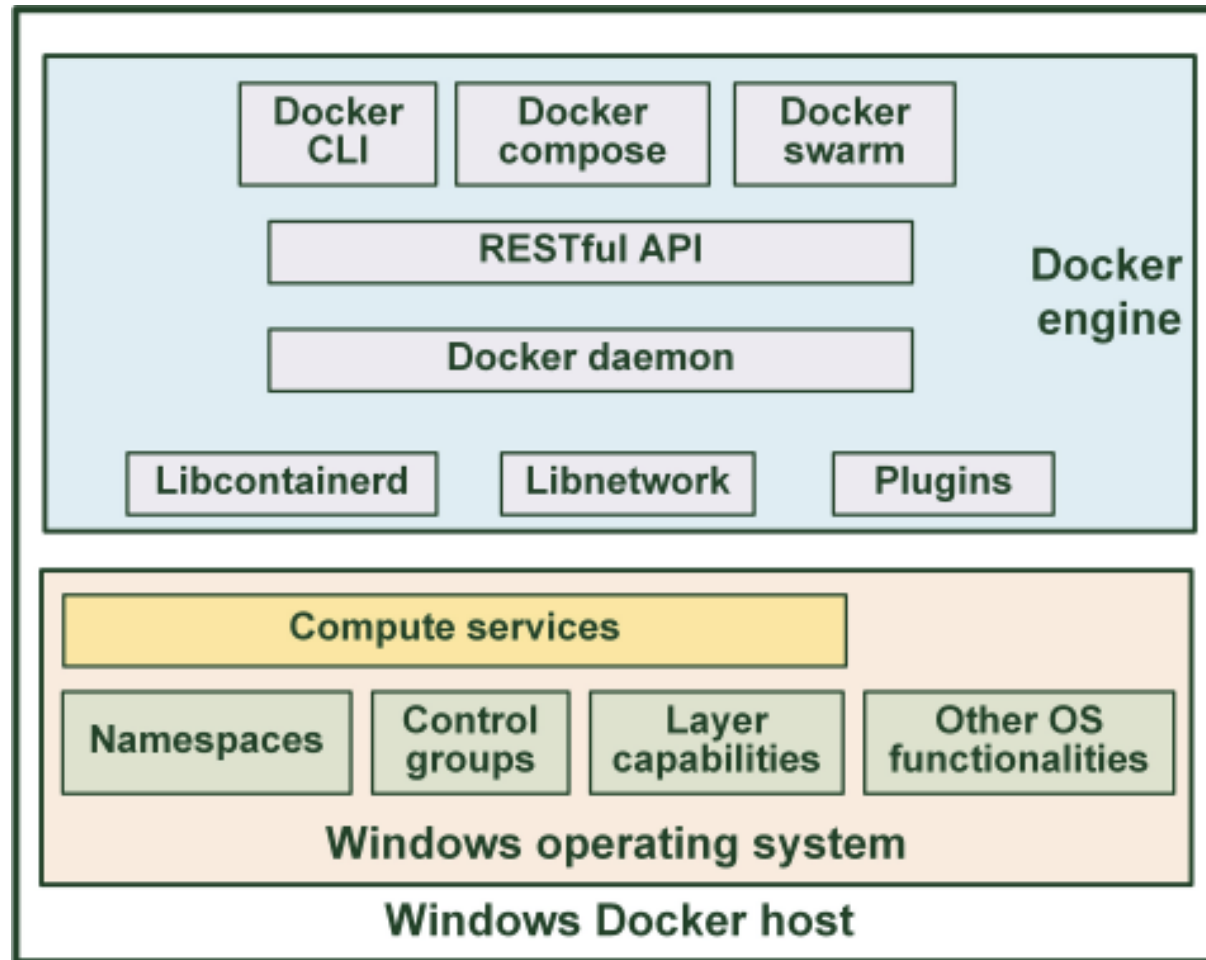
Architecture de l'hôte Linux Docker



Fondamentaux de Docker

Docker Architecture

- Architecture de l'hôte Docker Windows

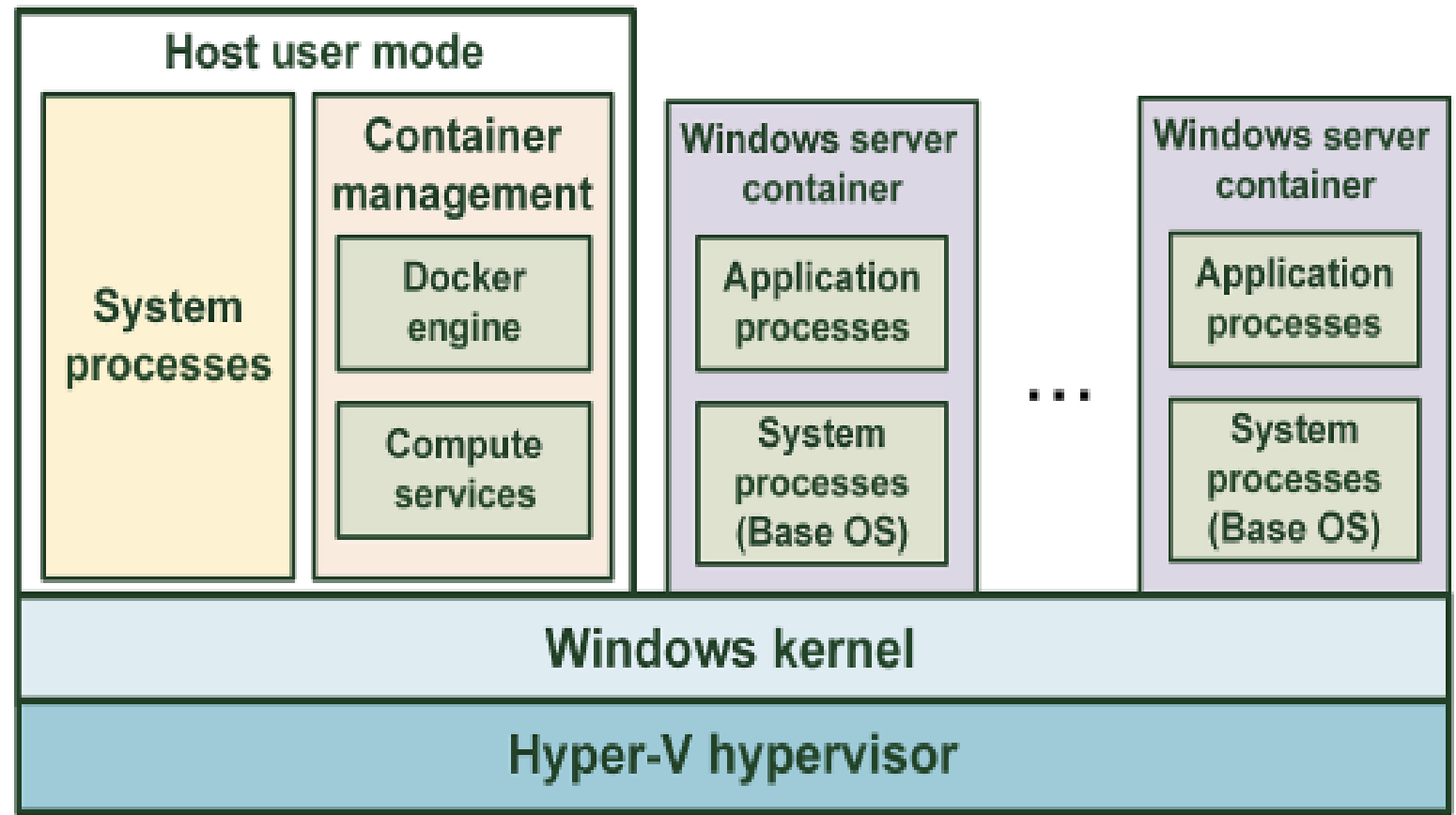


Fondamentaux de Docker

Docker Architecture

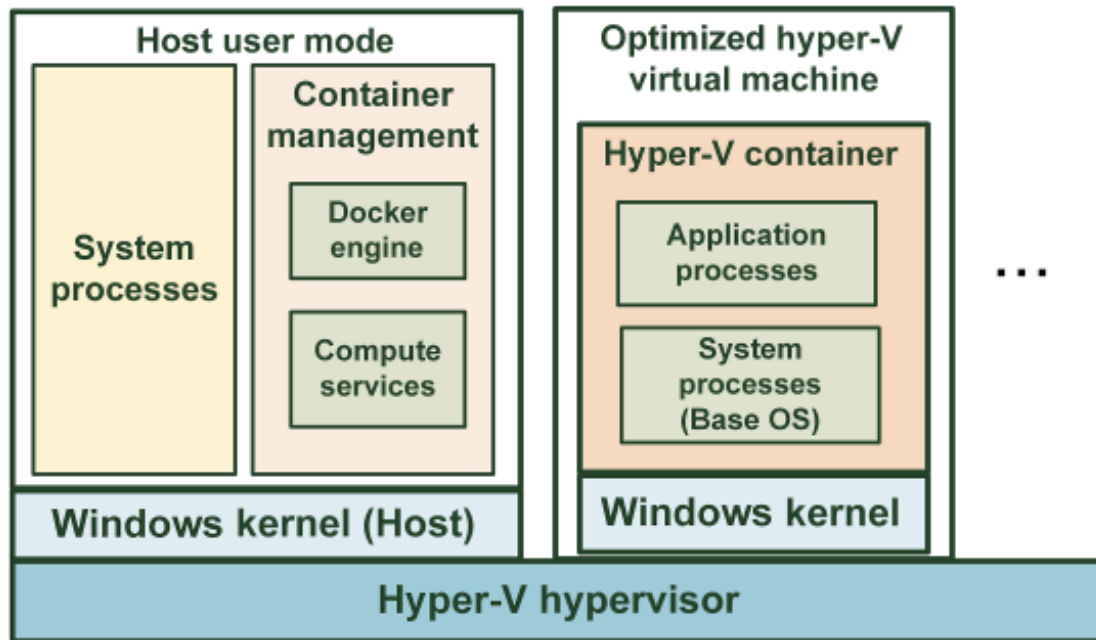
- Architecture des conteneurs Windows Server

- Au sein du système d'exploitation Windows, il existe deux modes : le **mode utilisateur** et le **mode noyau (kernel)**.
- **WSC**
- **DLL**

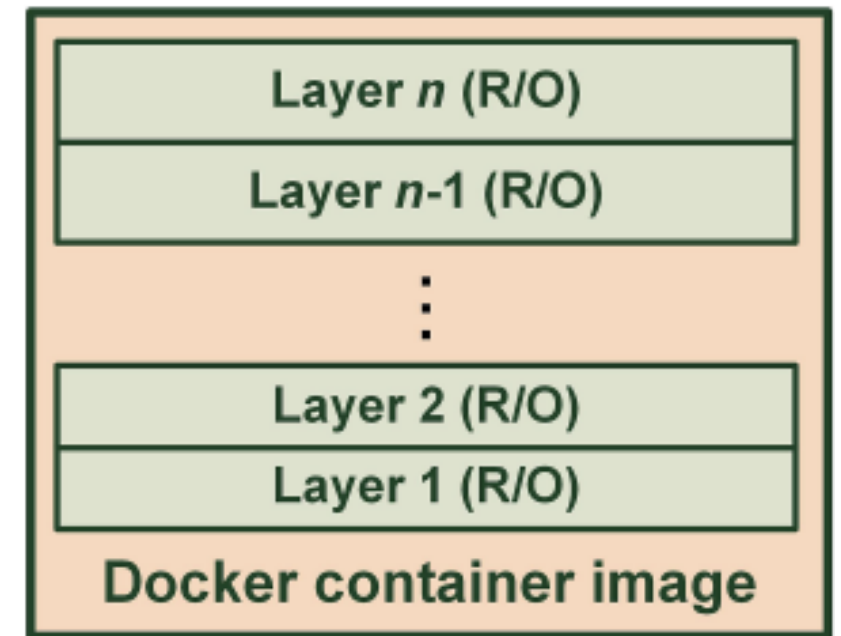


Fondamentaux de Docker

Docker Architecture



Architecture des conteneurs Hyper-V



Anatomie d'une image de conteneur Docker

Fondamentaux de Docker

Docker Architecture

```
# use ubuntu:latest as the base OS image
FROM ubuntu:latest
# install python
RUN apt-get update && apt-get install -y python
# copy the app from the host into the container
COPY hello.py /hello.py
# set starting command for the container
CMD ["python", "/hello.py"]
```

Un exemple de fichier Docker

```
mhhung@imrc-host1:~/build_pythonapp$ ls
Dockerfile hello.py
mhhung@imrc-host1:~/build_pythonapp$ docker image build -t pythonapp:latest .
Sending build context to Docker daemon 3.072kB
Step 1/4 : FROM ubuntu:latest
----> 9140108b62dc
Step 2/4 : RUN apt-get update && apt-get install -y python
----> Using cache
----> 4d54c3e75a3b
Step 3/4 : COPY hello.py /hello.py
----> Using cache
----> 8be88281b298
Step 4/4 : CMD ["python", "/hello.py"]
----> Using cache
----> 08bed6b197a4
Successfully built 08bed6b197a4
Successfully tagged pythonapp:latest
```

The process of building an image

docker image build t pythonapp latest

docker image history image name

docker images

mhhung@imrc-host1:~/build_pythonapp\$ docker image history pythonapp

	IMAGE	CREATED	CREATED BY	SIZE
layer from the CMD instruction	08bed6b197a4	2 days ago	/bin/sh -c #(nop) CMD ["python" "/hello.py"]	0B
layer from the COPY instruction	8be88281b298	2 days ago	/bin/sh -c #(nop) COPY file:3f8a14a336efe96f...	21B
layer from the RUN instruction	4d54c3e75a3b	2 days ago	/bin/sh -c apt-get update && apt-get install...	60.1MB
layer from the RUN instruction	9140108b62dc	10 days ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0B
layers from ubuntu base OS image	<missing>	10 days ago	/bin/sh -c mkdir -p /run/systemd && echo 'do...	7B
	<missing>	10 days ago	/bin/sh -c [-z "\$(apt-get indextargets)"]	0B
	<missing>	10 days ago	/bin/sh -c set -xe && echo '#!/bin/sh' > /...	811B
	<missing>	10 days ago	/bin/sh -c #(nop) ADD file:da80f59399481ffc3...	72.9MB

L'histoire de pythonapp :dernière image

Fondamentaux de Docker

Docker Architecture

```
# use ubuntu:18.04 as the base OS image
FROM ubuntu:18.04
# Instructions for installing nginx web server
RUN apt update && apt install -y nginx ...

# Instructions for coping files of the web app

# Expose Ports for the web app
EXPOSE 80 443
# define the starting command for the container
CMD ["/start.sh"]
```

Un raccourci de Dockerfile pour construire une image conteneur Linux

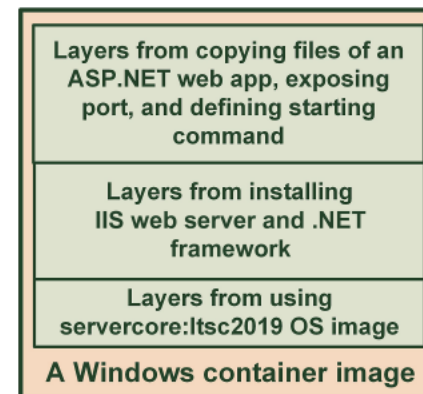
```
mhhung@imrc-host1:~/build_pythonapp$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
pythonapp           latest             08bed6b197a4       2 days ago         133MB
ubuntu              latest             9140108b62dc       10 days ago        72.9MB
```

Les tailles de deux images conteneurs Linux

```
# escape=`
# use servercore:ltsc2019 as the base OS image
FROM mcr.microsoft.com/windows/servercore:ltsc2019
# Instructions for installing IIS web server and .NET Framework
RUN powershell -Command `
    Add-WindowsFeature Web-Server; `
    ...
# Instructions for coping files of the web app

# Expose Port for the web app
EXPOSE 80
# define the starting command for the container
CMD ["/start.ps1"]
```

Un raccourci de Dockerfile pour construire un Image du conteneur Windows



Les couches empilées de l'image conteneur Windows construite par le fichier Docker dans la figure du milieu

Fondamentaux de Docker

Docker Architecture

Conteneur Docker

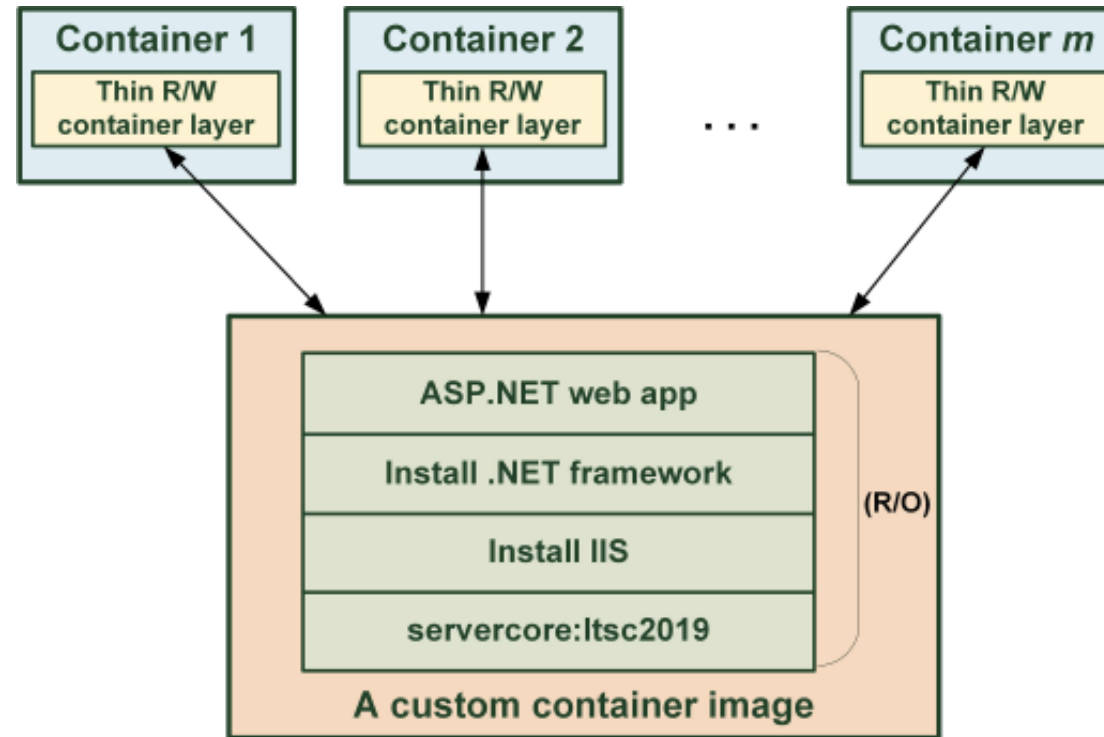
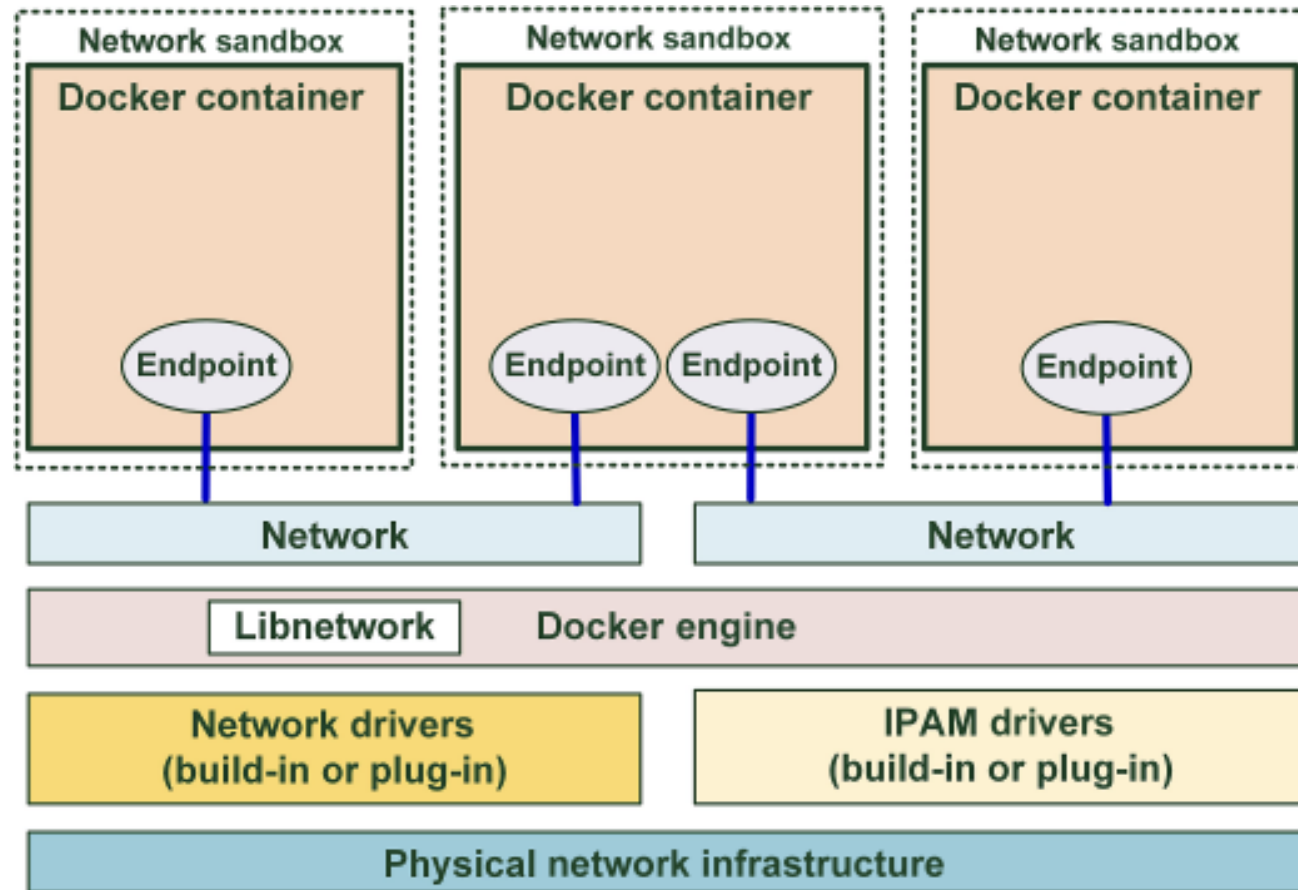


Illustration de nombreux conteneurs partageant les mêmes calques d'image

Fondamentaux de Docker

Docker Architecture

Modèle de réseau conteneur

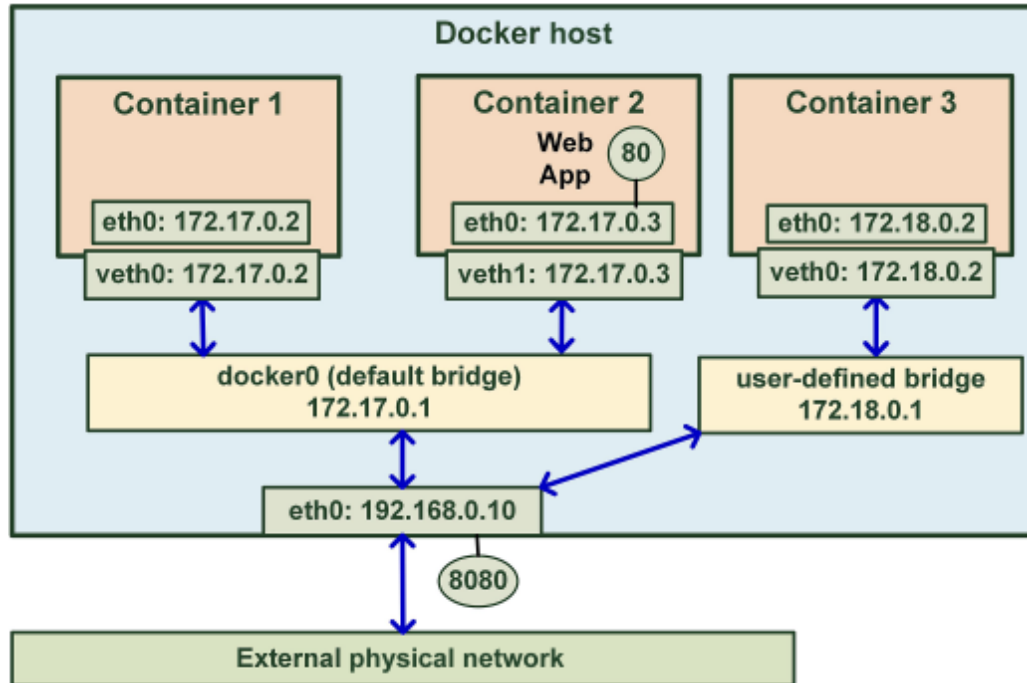


Architecture du modèle de réseau conteneur (CNM) pour conteneurs Linux

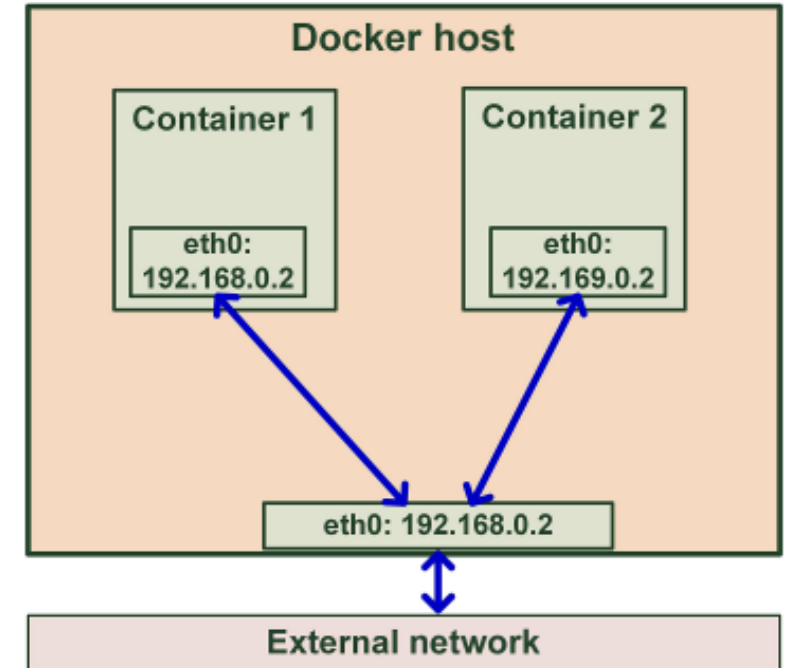
Fondamentaux de Docker

Docker Architecture

Réseau Docker



Architecture du réseau de ponts.

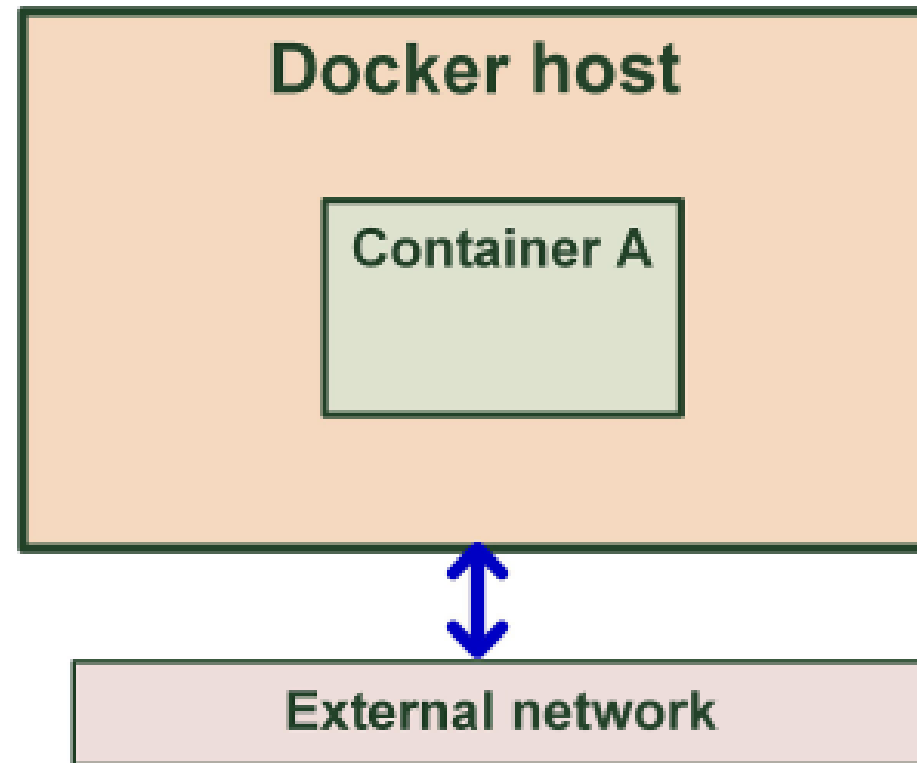


Architecture du réseau hôte.

Fondamentaux de Docker

Docker Architecture

- Aucun réseau
- Réseau superposé
- CNM pour conteneurs Windows

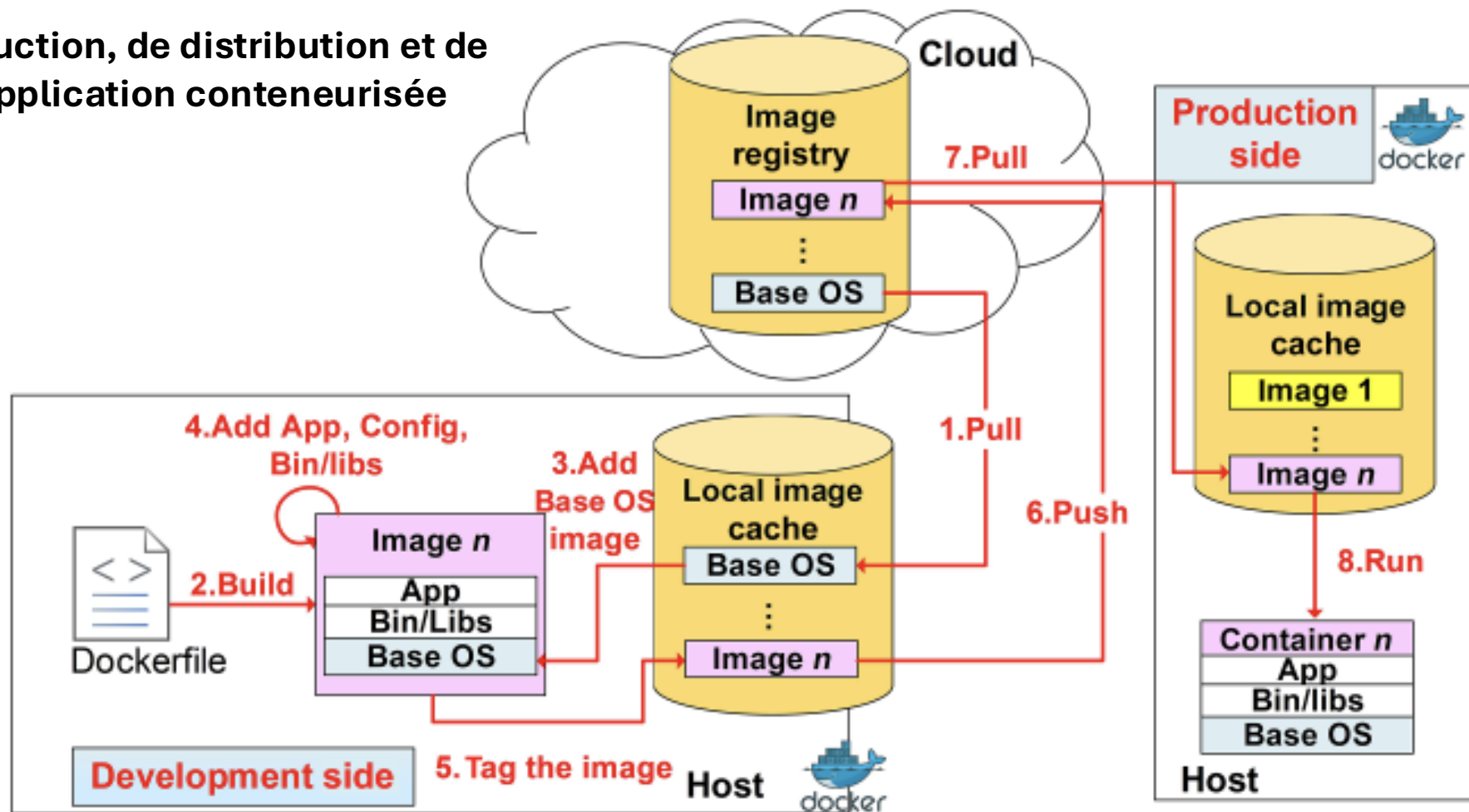


Architecture de non-réseau

Fondamentaux de Docker

Docker Architecture

Processus de construction, de distribution et de déploiement d'une application conteneurisée



Workflow de création, expédition et déploiement d'une application conteneurisée

Fondamentaux de Docker

Docker Architecture

Déploiement d'un conteneur Docker exécutant une application Linux

```
# use nginx as the base OS image, which includes Ubuntu OS and nginx environment
FROM nginx

# copy the web app from the host into the container
COPY . /usr/share/nginx/html/

# expose the port for web app
EXPOSE 80
```

Un exemple de Dockerfile pour créer une image d'application web Linux.

```
administrator@k8s-ubuntu-2:~/example-linux$ docker tag example-linux imrc/example-linux
administrator@k8s-ubuntu-2:~/example-linux$ docker push imrc/example-linux
The push refers to repository [docker.io/imrc/example-linux]
bda8becdd1d4: Pushed
908cf8238301: Mounted from library/nginx
eabfa4cd2d12: Mounted from library/nginx
60c688e8765e: Mounted from library/nginx
f431d0917d41: Mounted from library/nginx
07cab4339852: Mounted from library/nginx
latest: digest: sha256:ec982dd01326b49819508c42f5c6d19d251fb137c03c6d8cd9fab1d47ab1628f size: 1569
administrator@k8s-ubuntu-2:~/example-linux$
```

Image has been pushed to a repository on Docker Hub.

Execution results of the docker tag and docker push commands

```
administrator@k8s-slave-ubuntu-1:~$ docker pull imrc/example-linux
Using default tag: latest
latest: Pulling from imrc/example-linux
Digest: sha256:ec982dd01326b49819508c42f5c6d19d251fb137c03c6d8cd9fab1d47ab1628f
Status: Image is up to date for imrc/example-linux:latest
Image has been pulled from a repository on Docker Hub to another Docker host.
administrator@k8s-slave-ubuntu-1:~$
```

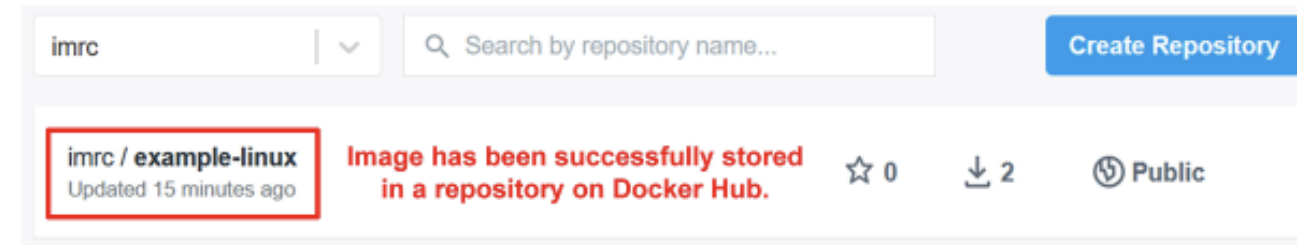
Résultat d'exécution de la commande docker pull

```
administrator@k8s-ubuntu-2:~/example-linux$ docker build . -t example-linux
Sending build context to Docker daemon 3.072kB
Step 1/3 : FROM nginx
----> 7e4d58f0e5f3
Step 2/3 : COPY . /usr/share/nginx/html/
----> Using cache
----> 7df05e2370ef
Step 3/3 : EXPOSE 80
----> Using cache
----> 5c0a243dd242
Successfully built 5c0a243dd242
Successfully tagged example-linux:latest
Image has been successfully built and tagged.
administrator@k8s-ubuntu-2:~/example-linux$
```

Steps of building image according to Dockerfile

Image has been successfully built and tagged.

Construire des étapes de l'image de l'application web Linux.



Screenshot showing that the “imrc/example-linux” image has been stored on Docker Hub.

```
administrator@k8s-slave-ubuntu-1:~$ docker run -p 8080:80 imrc/example-linux
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
Run a container from the pulled image.
```

Résultat d'exécution de la commande docker run.

Fondamentaux de Docker

Docker Architecture

Déploiement d'un conteneur Docker exécutant une application Windows



Hello, World!

This is an example of a simple HTML page with one paragraph.

Screenshot displaying the home page of the running containerized Linux web application.

```
C:\example-windows>docker build . -t imrc/example-windows
Sending build context to Docker daemon 5.154MB
Step 1/4 : FROM mcr.microsoft.com/dotnet/core/aspnet:3.1
--> 8689f698f6dc
Step 2/4 : WORKDIR /app
--> Using cache
--> f05ea1ee73a6
Step 3/4 : COPY /app ./
--> Using cache
--> 0680f4725864
Step 4/4 : ENTRYPOINT ["dotnet", "aspnetapp.dll"]
--> Using cache
--> 784112b991e0
Successfully built 784112b991e0
Successfully tagged imrc/example-windows:latest
```

Steps of building image according to Dockerfile

Image has been successfully built and tagged.

Building steps of the Windows web application image

```
# use ASP.NET 3.1 as the base OS image
FROM mcr.microsoft.com/dotnet/core/aspnet:3.1
```

```
# define /app as the current working directory
WORKDIR /app
```

```
# copy the web app from the host into the container
COPY /app ./
```

```
# set starting command for the container
ENTRYPOINT ["dotnet", "aspnetapp.dll"]
```

An example Dockerfile for building a Windows web application image

```
C:\example-windows>docker push imrc/example-windows
The push refers to repository [docker.io/imrc/example-windows]
bfeadbeca9c4: Pushed
4d58f1f94d05: Pushed
274ce4ac059f: Pushed
30f8b9916e17: Pushed
6f091ca5a20d: Pushed
784b3dcf1e52: Pushed
1f363421b3a5: Pushed
aebacb8b0709: Pushed
b7a95b530b4e: Pushed
604daa52528f: Skipped foreign layer
latest: digest: sha256:7e470abbd5da50611f79cc41612b1b3c9c5ef6b368996fb1a2febb67388013eb
size: 2587
```

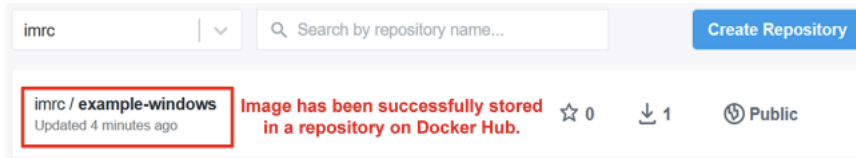
Image has been pushed to a repository on Docker Hub.

Execution result of the docker push command

Fondamentaux de Docker

Docker Architecture

Déploiement d'un conteneur Docker exécutant une application Windows



Capture d'écran montrant que l'image « imrc/example-windows » a été stockée sur Docker Hub

```
C:\>docker run -p 8080:80 imrc/example-windows
warn: Microsoft.AspNetCore.DataProtection.Repositories.FileSystemXmlRepository[60]
      Storing keys in a directory 'C:\Users\ContainerUser\AppData\Local\ASP.NET\DataProtection-Keys' that may not be persisted outside of the container. Protected data will be unavailable when container is destroyed.
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: http://[::]:80
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\app
```

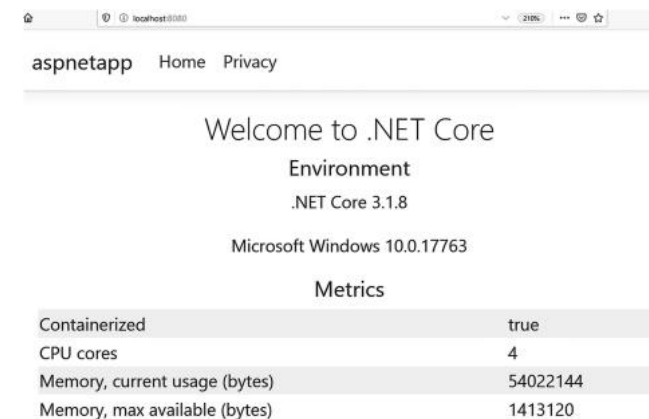
Run a container from the pulled image.

Résultat d'exécution de la commande docker run.

```
C:\>docker pull imrc/example-windows
Using default tag: latest
latest: Pulling from imrc/example-windows
Digest: sha256:7e470abbd5da50611f79cc41612b1b3c9c5ef6b368996fb1a2febb67388013eb
Status: Image is up to date for imrc/example-windows:latest
docker.io/imrc/example-windows:latest
```

Image has been pulled from a repository on Docker Hub to another Docker Host.

Résultat d'exécution de la commande pull Docker sur un hôte Docker Windows.



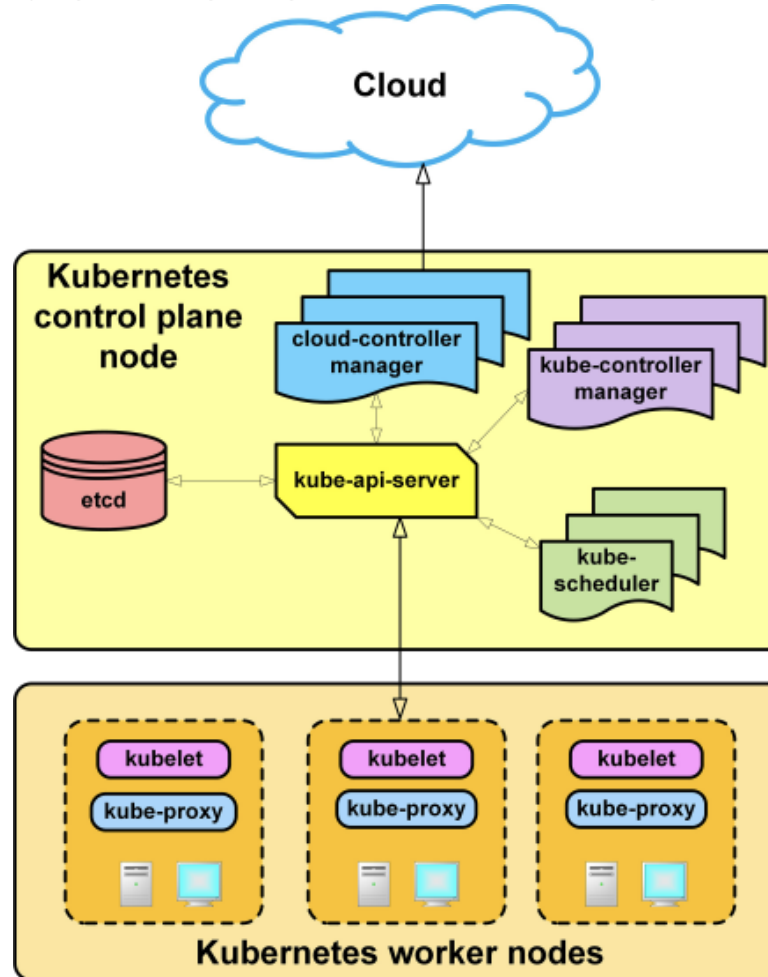
Capture d'écran affichant la page d'accueil de l'application web conteneurisée Windows en cours.

Fondamentaux de Kubernetes

Fondamentaux de Kubernetes

Kubernetes Architecture

- Fondements de Kubernetes
 - Kubernetes Architecture

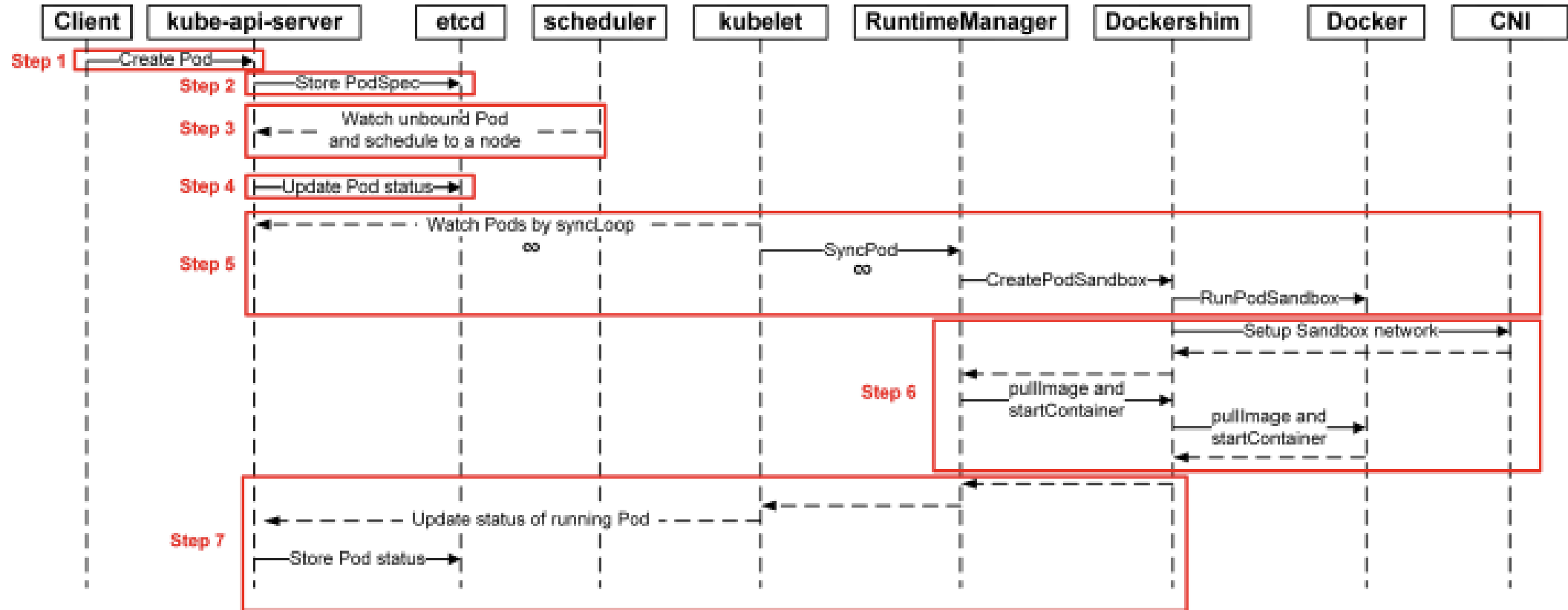


Architecture de Kubernetes. Source : Reproduit avec la permission de la Réf. [7] ; © 2020 Les auteurs Kubernetes, utilisés sous CC BY 4.0.

Fondamentaux de Kubernetes

Kubernetes Architecture

- Fondements de Kubernetes
- Kubernetes Architecture



Processus de création d'un pod. Source : Reproduit avec la permission de la Réf. [7] ; © 2020 Les auteurs Kubernetes, utilisés sous CC BY 4.0.

Fondamentaux de Kubernetes

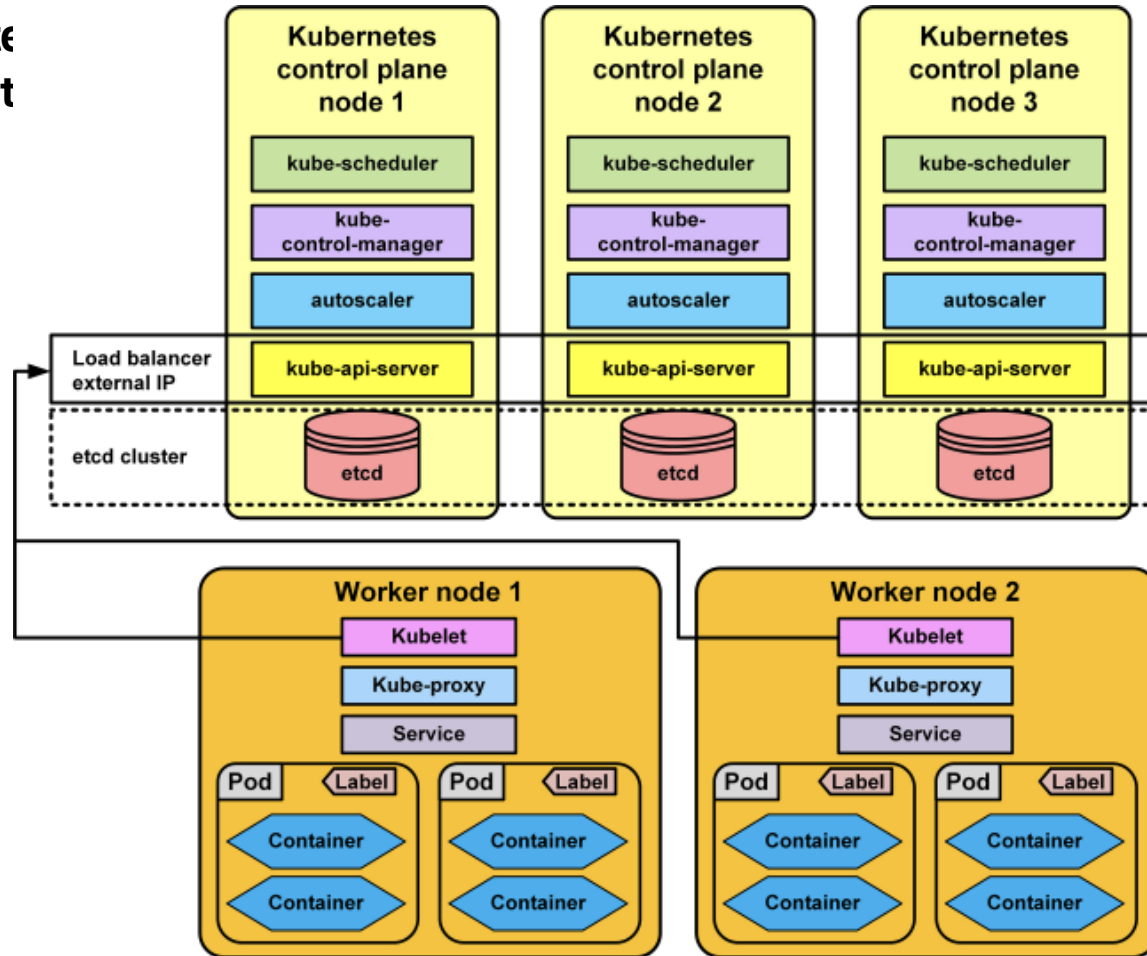
Kubernetes Architecture

- **Fondements de Kubernetes**
- **Architecture Kubernetes : Objets Kubernetes**
 - **Pods**
 - **Label and Label Selector**
 - **Service**
 - **Job**

Fondamentaux de Kubernetes

Kubernetes Architecture

- Fundamentals of Kubernetes
 - Kubernetes Architect

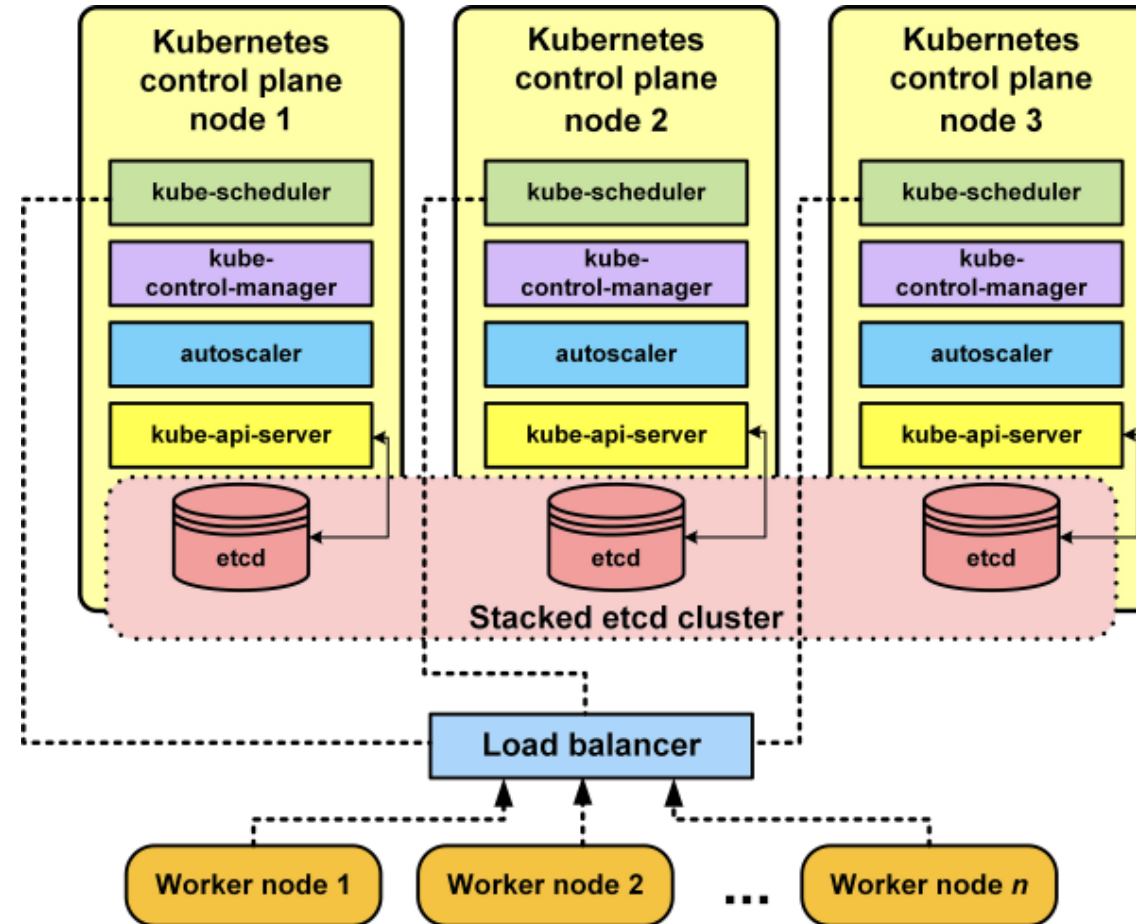


Architecture système avec trois nœuds de plan de contrôle

Fondamentaux de Kubernetes

Kubernetes Architecture

- Fundamentals of Kubernetes
 - Kubernetes Architecture

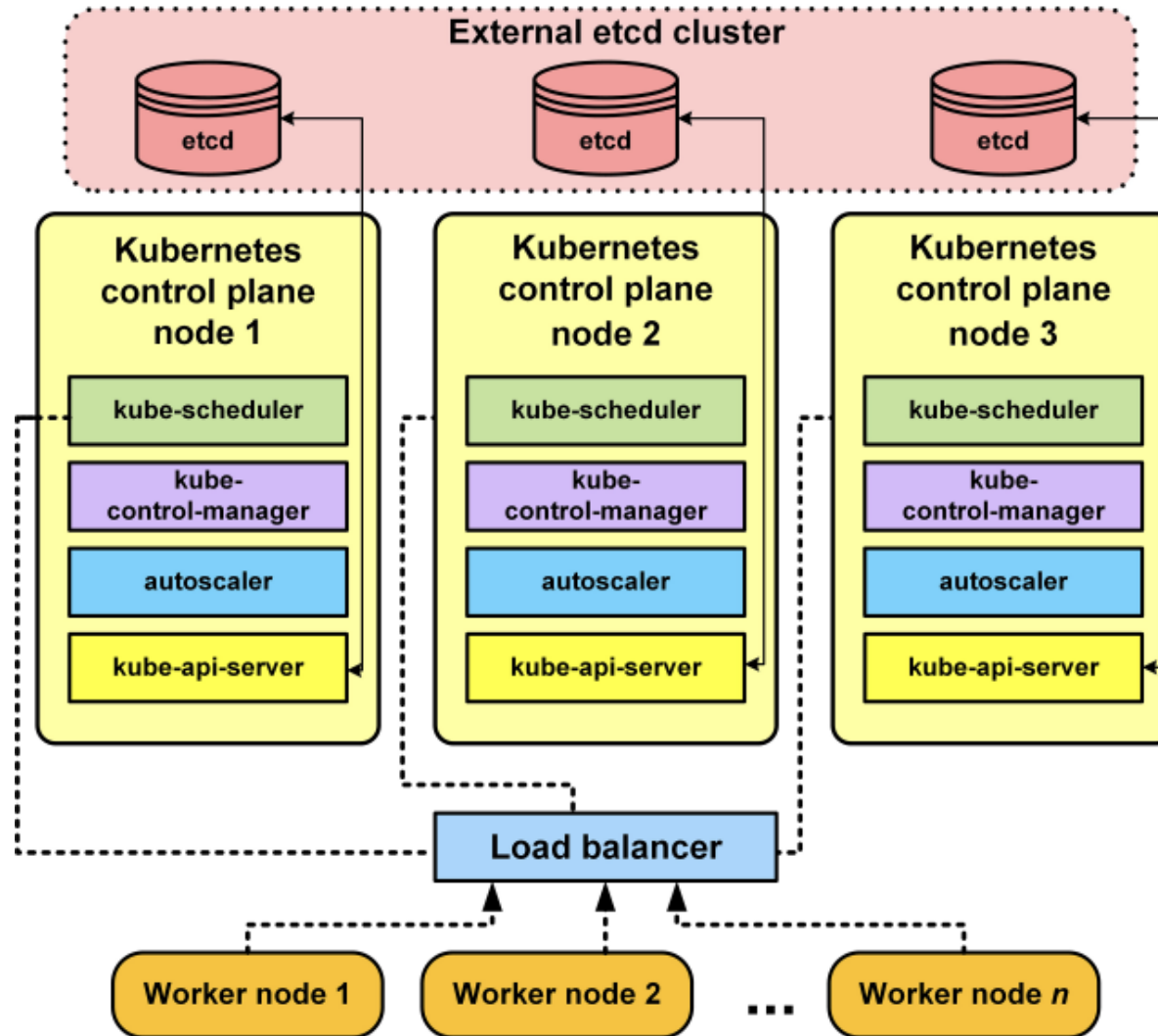


Architecture système avec la topologie etcd empilée

Fondamentaux de Kubernetes

Kubernetes Architecture

- Fundamentals of Kubernetes
 - Kubernetes Architecture

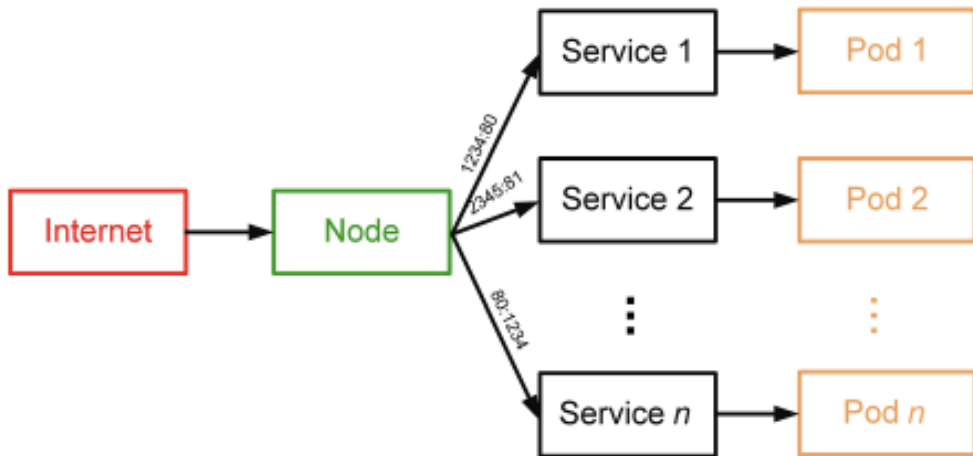


Architecture système avec la topologie etcd externe

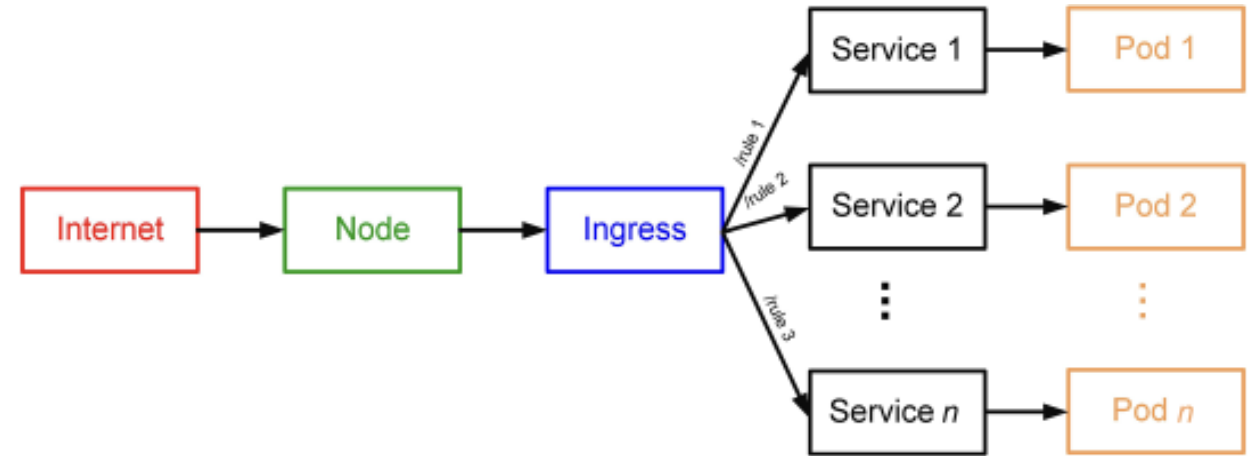
Fondamentaux de Kubernetes

Kubernetes Architecture

- Fundamentals of Kubernetes
 - Kubernetes Architecture



Architecture système sans entrée (Ingress)

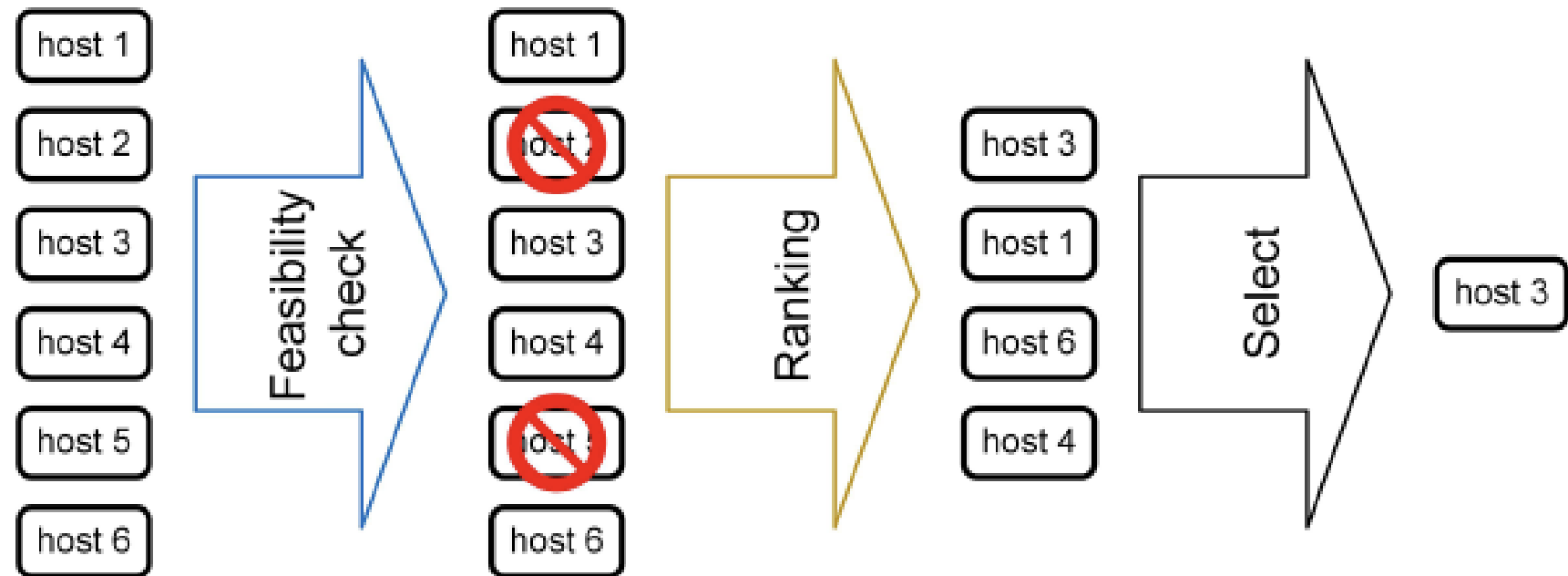


Architecture système avec entrée

Fondamentaux de Kubernetes

Kubernetes Architecture

- Fundamentals of Kubernetes
 - Replication
 - Scheduler
 - Autoscaling



Fondamentaux de Docker

Docker Architecture

- Illustrative Applications of Kubernetes

```
aaa@ubuntu:~$ sudo kubectl get pods --all-namespaces
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-6dcc67dcbc-nxbfj	1/1	Running	0	9m51s
kube-system	coredns-6dcc67dcbc-w2j6x	1/1	Running	0	9m51s
kube-system	etcd-ubuntu	1/1	Running	0	8m54s
kube-system	kube-apiserver-ubuntu	1/1	Running	0	8m43s
kube-system	kube-controller-manager-ubuntu	1/1	Running	0	9m6s
kube-system	kube-flannel-ds-amd64-zn5bl	1/1	Running	0	9m52s
kube-system	kube-proxy-rpk47	1/1	Running	0	9m52s
kube-system	kube-scheduler-ubuntu	1/1	Running	0	8m46s
kubernetes-dashboard	dashboard-metrics-scraper-7f669d7fbb-fwxwj	1/1	Running	0	4m19s
kubernetes-dashboard	kubernetes-dashboard-5845bf99b5-wwhz9	1/1	Running	0	4m19s

Statut de prêt du nœud du plan de contrôle

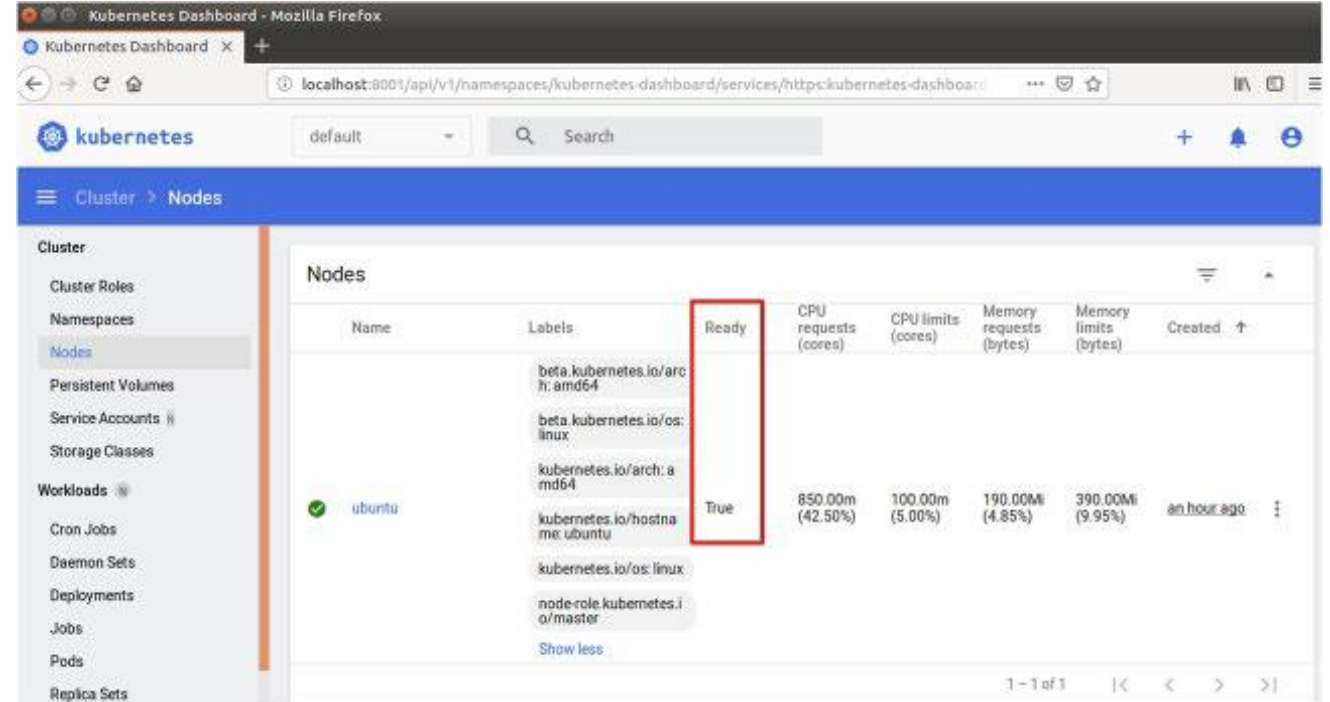


Tableau de bord pendant que le nœud du plan de contrôle est prêt

Fondamentaux de Docker

Docker Architecture

- Illustrative Applications of Kubernetes

```
aaa@ubuntu:~$ sudo kubeadm token create --print-join-command  
[sudo] password for aaa:  
kubeadm join 192.168.132.128:6443 --token 1gd1jy.j4j4p73gaxa8vhq1 --discover  
y-token-ca-cert-hash sha256:6a924f6c462580a94c710c097b97be129bd7f7f0e85f3bf23334  
77b441b35d7b  
aaa@ubuntu:~$
```

Génération du jeton de jointure du plan de contrôle

```
aaa@ubuntu:~$ sudo kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ubuntu	Ready	master	95m	v1.14.10
worker1	Ready	<none>	10m	v1.14.10
worker2	Ready	<none>	50s	v1.14.10

```
aaa@ubuntu:~$
```

Status of the cluster shown by kubectl.

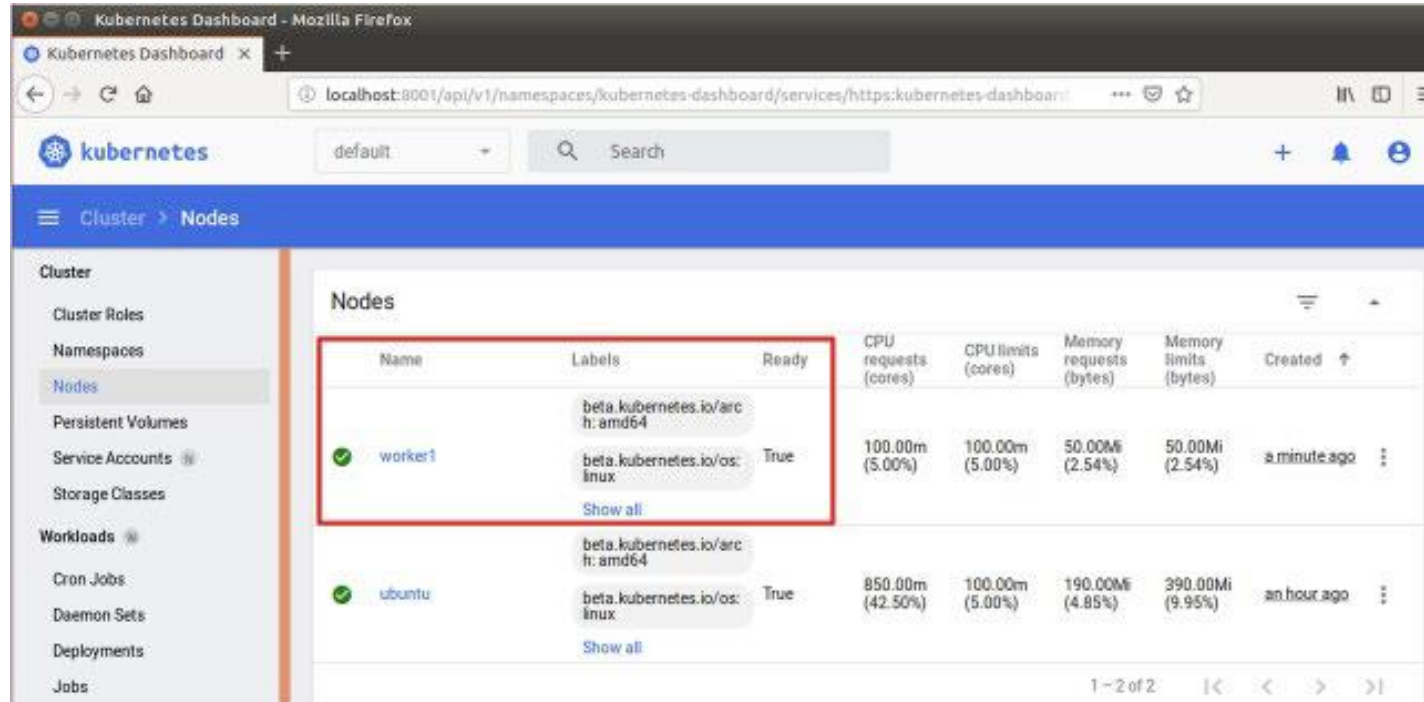
```
aaa@worker1:~/Desktop$ sudo kubeadm join 192.168.132.128:6443 --token 1gd1jy.j4j  
4p73gaxa8vhq1 --discovery-token-ca-cert-hash sha256:6a924f6c462580a94c710c09  
7b97be129bd7f7f0e85f3bf2333477b441b35d7b  
sudo: unable to resolve host worker1  
[preflight] Running pre-flight checks  
[WARNING IsDockerSystemdCheck]: detected "cgroupfs" as the Docker cgroup  
driver. The recommended driver is "systemd". Please follow the guide at https:/  
/kubernetes.io/docs/setup/cri/  
[WARNING SystemVerification]: this Docker version is not on the list of  
validated versions: 19.03.12. Latest validated version: 18.09  
[WARNING Hostname]: hostname "worker1" could not be reached  
[WARNING Hostname]: hostname "worker1": lookup worker1 on 8.8.8.8:53: no  
such host  
[preflight] Reading configuration from the cluster...  
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system g  
et cm kubeadm-config -oyaml'
```

Worker node joining the cluster by the join token

Fondamentaux de Docker

Docker Architecture

- Illustrative Applications of Kubernetes



The screenshot shows the Kubernetes Dashboard in a web browser. The 'Nodes' page is active, displaying a table of cluster nodes. The 'worker1' node is highlighted with a red box. The table columns are: Name, Labels, Ready, CPU requests (cores), CPU limits (cores), Memory requests (bytes), Memory limits (bytes), and Created.

Name	Labels	Ready	CPU requests (cores)	CPU limits (cores)	Memory requests (bytes)	Memory limits (bytes)	Created
worker1	beta.kubernetes.io/arch: amd64 beta.kubernetes.io/os: linux	True	100.00m (5.00%)	100.00m (5.00%)	50.00Mi (2.54%)	50.00Mi (2.54%)	a minute ago
ubuntu	beta.kubernetes.io/arch: amd64 beta.kubernetes.io/os: linux	True	850.00m (42.50%)	100.00m (5.00%)	190.00Mi (4.85%)	390.00Mi (9.95%)	an hour ago

Statut de l'ouvrier1 affiché dans le tableau de bord.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpd-deployment
labels:
  app: httpd
spec:
  replicas: 3
  selector:
    matchLabels:
      app: httpd
  template:
    metadata:
      labels:
        app: httpd
    spec:
      containers:
        - name: httpd
          image: httpd
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: httpd
labels:
  app: httpd
spec:
  ports:
    - port: 80
    targetPort: 80
  selector:
    app: httpd
```

Annotations in the image:

- `kind: Deployment`: Type of Kubernetes component (Deployment)
- `name: httpd-deployment`: Deployment name
- `replicas: 3`: Number of replica
- `labels: app: httpd`: Labels attached to this component
- `image: httpd`: Image used for this deployment
- `containerPort: 80`: Port used by this deployment

Exemple de fichier YAML

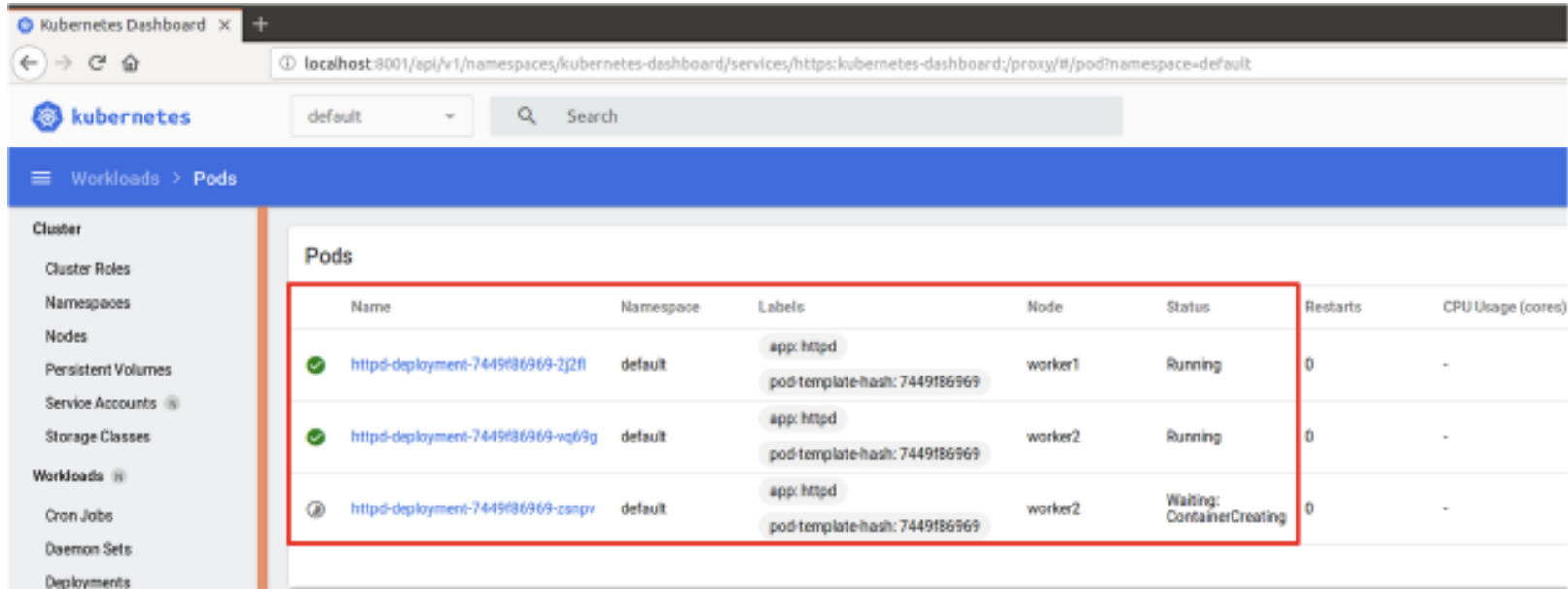
```
aaa@ubuntu:~$ cd Desktop/
aaa@ubuntu:~/Desktop$ sudo kubectl apply -f example.yaml
deployment.apps/httpd-deployment created
service/httpd created
aaa@ubuntu:~/Desktop$
```

Déploiement du service httpd en appliquant example.yaml

Fondamentaux de Docker

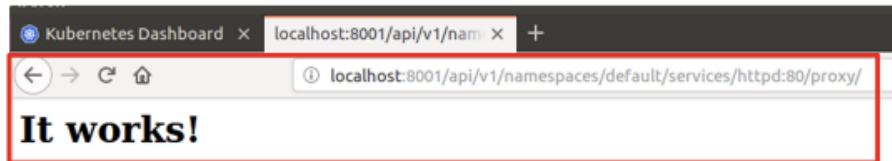
Docker Architecture

- Illustrative Applications of Kubernetes



Name	Namespace	Labels	Node	Status	Restarts	CPU Usage (cores)
✔ httpd-deployment-7449f86969-2j2fl	default	app: httpd pod-template-hash: 7449f86969	worker1	Running	0	-
✔ httpd-deployment-7449f86969-vq69g	default	app: httpd pod-template-hash: 7449f86969	worker2	Running	0	-
⌚ httpd-deployment-7449f86969-zsnpv	default	app: httpd pod-template-hash: 7449f86969	worker2	Waiting: ContainerCreating	0	-

Statut des Pods affiché dans le tableau de bord



Capture d'écran affichant un service httpd fonctionnel

Fondamentaux de Docker

Docker Architecture

- **Illustrative Applications of Kubernetes**

Kubernetes offre la capacité de déploiement, d'échelon et de gestion automatiques des charges de travail et services intégrés. Il pouvait orchestrer efficacement les ressources du cluster pour satisfaire les besoins en ressources des Pods et services. Cette section a introduit les fondamentaux et les principes opérationnels essentiels de Kubernetes. Il existe d'autres concepts et documents de Kubernetes sur le site web et les documents en ligne, qui valent la peine d'être explorés.

Fin du chapitre 3